

BINX TECH • BACKEND DEVELOPMENT INTERNSHIP PROGRAM (.NET)

WEEK 1

ONBOARDING, C# FUNDAMENTALS & GIT

Setting up a professional .NET development environment and building the core C# fundamentals every backend service in this program will be built on.

Week 1 opens Phase 1 of the program. Interns install and configure the .NET SDK, get comfortable with the dotnet CLI and a real IDE, and work through core C# language fundamentals — types, object-oriented programming, collections, and LINQ — before touching ASP.NET Core or any web framework. The week closes with a proper Git and GitHub workflow, the same one used for every sprint in Phase 3.

PHASE 1 • 40 HOURS • 5 TRAINING DAYS • FOUNDATIONS

Prepared by BinX Tech

Palestine | Nablus

BinX

TECH



1. Week 1 Overview

Field	Detail
Week	Week 1 of 10 — Phase 1: Onboarding & Foundations
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 1-2 combined)
Format	On-site / Remote / Hybrid — work committed to a personal GitHub repository
Focus	Onboarding, .NET SDK setup, C# fundamentals (OOP, collections), Git basics
Prerequisite	None — Week 1 is the program's entry point
Mentor Supervision	Kickoff orientation on Day 1; review of the Week 1 pull request on Day 5

2. Week 1 Learning Objectives

- Install, configure, and verify a working .NET SDK development environment.
- Navigate the dotnet CLI to create, build, and run a .NET console application.
- Apply core C# fundamentals: types, control flow, and nullable reference types.
- Model a small domain using classes, records, interfaces, and object-oriented principles.
- Query collections using LINQ and write basic asynchronous code with async/await.
- Use Git and a feature-branch GitHub workflow to commit, branch, and open a pull request.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Program orientation; .NET development environment setup
Day 2	8 hrs	C# fundamentals I: types, variables, and control flow
Day 3	8 hrs	C# fundamentals II: object-oriented programming
Day 4	8 hrs	C# fundamentals III: collections and LINQ basics
Day 5	8 hrs	Git and GitHub workflow; Week 1 synthesis



4. Day-by-Day Curriculum

Day 1 — Program Orientation & .NET Development Environment Setup — 8 hours

Learning Objectives

- Understand the program's four phases and how Weeks 1-10 build on each other.
- Install and verify a working .NET SDK.
- Navigate the dotnet CLI to create, build, and run a console application.

Key Topics

- Program orientation: phases, deliverables, evaluation, mentor structure
- Installing and verifying the .NET SDK
- The dotnet CLI
- Choosing and configuring an IDE

Lesson Content

1.1 Program Orientation

Day 1 opens with a walkthrough of the full 400-hour program: the four phases, the weekly cadence, the mentor check-in structure, and what “done” looks like for the Phase 3 capstone project. Interns leave with a GitHub account ready for the program, a Notion project tracker, and a clear picture of the six capstone project types they will choose between in Week 5.

1.2 Installing and Verifying the .NET SDK

The .NET SDK bundles the runtime, the compiler, and the dotnet CLI into a single installer, available for Windows, macOS, and Linux. After installation, running “dotnet --version” confirms the SDK is correctly on the system PATH — a mismatched or missing version here is the single most common blocker in Day 1 setup, and worth verifying before moving on to anything else.

```
$ dotnet --version
8.0.100

$ dotnet --list-sdks
8.0.100 [/usr/local/share/dotnet/sdk]
```

1.3 The dotnet CLI

The dotnet CLI is the primary interface for creating, building, running, and testing .NET projects — “dotnet new console” scaffolds a new console application, “dotnet run” compiles and executes it in one step, and “dotnet build” compiles without running. Every project built across this entire program, from today's first console app through the Phase 3 capstone API, starts with these same handful of commands.

1.4 Choosing and Configuring an IDE

Visual Studio Code with the C# Dev Kit extension is lightweight and cross-platform, while Visual Studio (Windows/Mac) offers a more integrated, batteries-included experience with a visual debugger and designer tools — either is a valid choice for this program, and the setup steps in this curriculum work with both. What matters is configuring IntelliSense and the debugger correctly today, since every following week depends on both working reliably.

Run “dotnet --version” and successfully build and run a “Hello World” console app before ending Day 1. A shaky environment discovered in Week 3 mid-project costs far more time than catching it now.



Hands-On Lab: Environment Setup & First Console App

1. Install the current .NET LTS SDK and confirm it with "dotnet --version".
2. Install and configure your chosen IDE (VS Code with C# Dev Kit, or Visual Studio) with debugging enabled.
3. Create a new console app with "dotnet new console -o HelloBinX" and run it with "dotnet run".
4. Modify the program to print your name and today's date, then rebuild and rerun it.
5. Create a GitHub account (if you don't have one) and a new repository named "[YourName] — BinX Backend Internship."

Tools Used

.NET SDK • Visual Studio Code (C# Dev Kit) or Visual Studio • Terminal • GitHub

Day 2 — C# Fundamentals I: Types, Variables & Control Flow — 8 hours

Learning Objectives

- Distinguish value types from reference types and explain why the difference matters.
- Use variables, type inference, and clear naming conventions idiomatically.
- Write control flow using if statements, switch expressions, and loops.

Key Topics

- Value types vs. reference types
- Variables, type inference, and naming
- Control flow: if, switch expressions, loops
- Nullable reference types

Lesson Content

2.1 Value Types vs. Reference Types

A value type (int, bool, struct) holds its data directly and is copied by value when assigned or passed to a method — changing a copy never affects the original. A reference type (class, string, array) holds a reference to data stored elsewhere, so assigning or passing it copies the reference, not the underlying data — both variables end up pointing at the same object. Misunderstanding this distinction is one of the most common sources of subtle bugs for developers new to C#.

2.2 Variables, Type Inference, and Naming

The "var" keyword lets the compiler infer a variable's type from its assigned value — "var count = 5;" is exactly as strongly typed as "int count = 5;", just less explicit to read. This program favors explicit types for public method signatures and return types (where clarity for a reader matters most) and allows "var" for local variables where the type is already obvious from the right-hand side.

2.3 Control Flow

Beyond the familiar if/else, C#'s switch expression (distinct from the older switch statement) returns a value directly from a pattern match, which reads more clearly than a chain of if/else branches when a value maps to one of several outcomes. Loops — for, foreach, while — each fit a different shape of problem: foreach for iterating a known collection, while for a condition-driven repeat, for when the index itself matters.

```
string DescribeGrade(int score) => score switch
{
    >= 90 => "Excellent",
    >= 70 => "Proficient",
    >= 50 => "Developing",
    -    => "Below Standard"
};
```

2.4 Nullable Reference Types

With nullable reference types enabled (the default for new .NET projects), the compiler tracks whether a reference type can be null and warns when a possibly-null value is used without a check — “string? name” explicitly allows null, while “string name” does not, and the compiler holds code to that promise. This catches an entire category of null-reference exceptions at compile time instead of at runtime, which is exactly the kind of production reliability this program requires by Week 9’s Definition of Done audit.

A compiler warning about a possibly-null reference is not noise to silence with “!” out of convenience. It’s the compiler catching a real bug before it ships — treat it as a bug report, not an obstacle.

Hands-On Lab: Types & Control Flow Practice

1. Write a console program with at least 3 value-type and 3 reference-type variables, printing each one’s type using “GetType()”.
2. Write a method that demonstrates the value-vs-reference copy behavior, printing before and after a mutation.
3. Write a grade-classifier method using a switch expression, covering at least 4 score ranges.
4. Write a small program that reads user input and handles a possibly-null value safely, with nullable reference types enabled.
5. Commit the day’s work to your GitHub repository with a clear, descriptive commit message.

Tools Used

.NET SDK • VS Code or Visual Studio • Git



Day 3 — C# Fundamentals II: Object-Oriented Programming — 8 hours

Learning Objectives

- Choose between a class, a record, and a struct for a given modeling need.
- Apply encapsulation using access modifiers and properties.
- Use inheritance and interfaces to model shared behavior and polymorphism.

Key Topics

- Classes, records, and structs
- Encapsulation and access modifiers
- Inheritance and interfaces
- Polymorphism in practice

Lesson Content

3.1 Classes, Records, and Structs

A class is the default choice for a reference type with identity and behavior — an entity whose “sameness” is about being the same object, not having the same data. A record is a reference type designed for immutable data with built-in value-based equality — two records with identical property values are considered equal, which fits data transfer objects and API request/response models well. A struct is a value type, appropriate for small, immutable data (a coordinate, a money amount) where copy-by-value behavior and avoiding heap allocation matter.

```
// Record: immutable, value-based equality — ideal for API DTOs
public record CreateUserRequest(string Email, string DisplayName);

// Class: identity-based, mutable state and behavior
public class UserAccount
{
    public Guid Id { get; }
    public string Email { get; private set; }
}
```

3.2 Encapsulation and Access Modifiers

Access modifiers (public, private, protected, internal) control what a class exposes to the outside world versus what it keeps as internal implementation detail. A well-encapsulated class exposes properties and methods that represent its behavior, while keeping the fields backing that behavior private — this is what allows the internal implementation to change later without breaking every piece of code that uses the class.

3.3 Inheritance and Interfaces

Inheritance lets a class share implementation with a base class (“Admin : User”), appropriate when a true is-a relationship exists. An interface defines a contract — a set of members a class promises to implement — without dictating any shared implementation, appropriate when unrelated classes need to guarantee the same capability (an “INotifiable” interface implemented by both an Order and a User class, for instance). Favoring interfaces over deep inheritance chains is generally the more flexible, more testable choice in modern C#.

3.4 Polymorphism in Practice

Polymorphism lets code written against a base type or interface work correctly with any derived type — a method accepting “INotifiable target” works identically whether it's actually handed an Order or a User, without needing to know which. This is the mechanism that lets this program's later ASP.NET Core services depend on interfaces rather than concrete implementations, which is the foundation dependency injection (covered in Week 2) is built on.



A common early mistake is reaching for inheritance by default. If the relationship between two types is really “can do the same thing” rather than “is fundamentally the same kind of thing,” an interface is almost always the better fit.

Hands-On Lab: Model a Small Domain

1. Design a small domain (e.g. a library, a task tracker, or an order system) with at least 2 related classes.
2. Add at least one record type for an immutable data transfer object in your domain.
3. Apply proper encapsulation: private backing fields, public properties, and a constructor that enforces valid initial state.
4. Define one interface implemented by two unrelated classes in your domain, demonstrating polymorphism with a method that accepts the interface type.
5. Commit the domain model to your GitHub repository.

Tools Used

.NET SDK • VS Code or Visual Studio • Git

Day 4 — C# Fundamentals III: Collections & LINQ Basics — 8 hours

Learning Objectives

- Choose the right collection type for a given data-access pattern.
- Query and transform collections using LINQ.
- Write and reason about basic asynchronous code with async/await.

Key Topics

- Choosing the right collection
- LINQ: method syntax and query syntax
- Async/await fundamentals
- Exception handling

Lesson Content

4.1 Choosing the Right Collection

A `List<T>` is the default general-purpose ordered collection. A `Dictionary<TKey, TValue>` is the right choice when data needs to be looked up by a key rather than scanned sequentially — an $O(1)$ lookup versus a `List`'s $O(n)$ search. A `HashSet<T>` is the right choice when uniqueness matters more than order, such as tracking which IDs have already been processed. Choosing the wrong collection for the access pattern is a common, easily avoidable source of unnecessary slowness.

4.2 LINQ: Method Syntax and Query Syntax

LINQ (Language Integrated Query) lets collections be filtered, transformed, and aggregated with a consistent, declarative syntax rather than hand-written loops. Method syntax (“`users.Where(u => u.IsActive).Select(u => u.Name)`”) chains operations fluently and is the more common style in production code; query syntax (“`from u in users where u.IsActive select u.Name`”) reads closer to SQL and can be clearer for complex joins. Both compile to the same underlying calls.

```
var activeAdminEmails = users
    .Where(u => u.IsActive && u.Role == "Admin")
    .OrderBy(u => u.DisplayName)
    .Select(u => u.Email)
    .ToList();
```




4.3 Async/Await Fundamentals

Marking a method “async” and using “await” on an I/O-bound operation (a database call, an HTTP request, a file read) lets the thread be released back to the pool while waiting, instead of blocking — essential for a backend API that needs to handle many concurrent requests efficiently. Every database and network call built in this program from Week 3 onward will be async, so getting comfortable with the pattern now, even in simple console examples, pays off immediately.

4.4 Exception Handling

A try/catch block should catch specific exception types where the calling code can meaningfully recover, rather than a blanket catch of the base Exception type that silently swallows every kind of failure. A finally block runs regardless of whether an exception occurred, appropriate for cleanup logic (closing a file, releasing a resource) that must always execute.

A try/catch that catches an exception and does nothing with it — or just logs “something went wrong” — is worse than not catching it at all, since it hides the real failure from whoever debugs it later. Catch specifically, and handle or rethrow deliberately.

Hands-On Lab: LINQ Queries & an Async Method

1. Create a List of at least 8 objects from your Day 3 domain model with varied property values.
2. Write 3 LINQ queries against the list: one filter, one projection, and one aggregation (Count, Sum, or Average).
3. Write an async method that simulates an I/O delay (Task.Delay) and returns a result, then await it from Main.
4. Wrap a risky operation (e.g. parsing user input) in a try/catch that catches a specific exception type and handles it meaningfully.
5. Commit the day's work to your GitHub repository.

Tools Used

.NET SDK • VS Code or Visual Studio • Git

Day 5 — Git & GitHub Workflow; Week 1 Synthesis — 8 hours

Learning Objectives

- Use Git's core commands to track and manage changes to a codebase.
- Follow a feature-branch workflow matching how the Phase 3 sprints will operate.
- Open a pull request with a clear description, the same standard used for every sprint task.

Key Topics

- Git fundamentals
- The feature-branch workflow
- Writing good commit messages
- Opening a pull request

Lesson Content

5.1 Git Fundamentals

Git tracks changes to a codebase as a sequence of commits, each a snapshot with a message describing what changed and why. The core loop — “git add” to stage changes, “git commit” to snapshot them, “git push” to sync with GitHub — is the same loop used on every professional team, and it's worth building the muscle memory for it now rather than under Phase 3's sprint deadlines.



5.2 The Feature-Branch Workflow

Rather than committing directly to the main branch, work happens on a dedicated feature branch ("git checkout -b feature/user-model"), keeping main always in a known-good, deployable state. This is exactly the workflow the acceptance criteria in every future sprint will require: code committed to the correct feature branch, with a pull request opened and reviewed before it merges into main.

```
$ git checkout -b feature/week1-oop-domain
$ git add .
$ git commit -m "Add domain model with User and Order classes"
$ git push -u origin feature/week1-oop-domain
```

5.3 Writing Good Commit Messages

"fix stuff" tells a future reader nothing; "Fix null reference exception when parsing empty order list" tells them exactly what changed and why it mattered. A good commit message describes the change in imperative mood ("Add," "Fix," "Refactor," not "Added" or "Fixing") and stays specific enough that someone scanning the commit history later can understand the project's evolution without opening every diff.

5.4 Opening a Pull Request

A pull request bundles a feature branch's commits into a single reviewable unit, with a description explaining what changed and why, ready for a mentor's structured code review — the same review process every Phase 3 sprint task will go through. Opening a PR even for solo Week 1 practice work builds the habit before it matters for a real teammate's review.

A pull request with the description "Week 1 stuff" gives a reviewer nothing to work with. Even a short, specific description — what this branch adds, and any open questions — makes review meaningfully faster and more useful.

Hands-On Lab: Git Workflow & Week 1 Close-Out

1. Initialize Git in your Week 1 project folder (if not already done) and confirm your GitHub remote is connected.
2. Create a feature branch, commit your Day 2-4 work to it with clear, specific commit messages, and push it to GitHub.
3. Open a pull request from your feature branch into main, with a description summarizing the week's work.
4. Request your mentor as a reviewer on the pull request.
5. Assemble a Week 1 summary in Notion: environment setup notes, the 3 C# exercises, and a link to your pull request — ready for the mentor check-in.

Tools Used

Git • GitHub • Notion

5. Week 1 Deliverables

- A verified, working .NET SDK development environment with a configured IDE.
- A console program demonstrating C# types, control flow, and nullable reference type handling.
- A small object-oriented domain model using classes, records, and at least one interface.
- LINQ queries and an async method demonstrated over a collection.
- A GitHub repository with a feature branch, clear commit history, and an opened pull request.
- A Week 1 summary document in Notion, ready for the mentor check-in.



6. Week 1 Evaluation Criteria

Scored by the assigned mentor at the Week 1 check-in using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of core concepts	Partial understanding, often confused	Explains most decisions clearly	Deep understanding, can teach it to others
C# & .NET usage	Basic types and classes only	Interfaces, generics, async/await used correctly	Nullable reference types, idiomatic C# throughout
Code quality & structure	Basic structure, inconsistencies	Clean, readable, consistent patterns	Production-grade, modular, well-organised
Git & GitHub workflow	Infrequent or unclear commits	Feature branch used, clear commit messages	Clean history, well-described pull request, proactive
Problem-solving & debugging	Struggles to resolve errors independently	Resolves most errors with minimal guidance	Debugs independently, explains root causes clearly
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 1

Category	Tools
Language & Runtime	.NET SDK (current LTS), C#
Development Environment	Visual Studio Code (C# Dev Kit) or Visual Studio
Version Control	Git, GitHub
Documentation	Notion (Week 1 summary, program tracker)

8. Notes & Best Practices

Week 1 is deliberately language- and tooling-heavy, with no web framework in sight yet — that's intentional. Every ASP.NET Core concept from Week 2 onward assumes fluency with the C# fundamentals practiced this week: types, OOP, LINQ, and async/await. The Git workflow set up on Day 5 is the exact same one used for every Phase 3 sprint, so the habit built now is not a one-off exercise — it's the actual process this program runs on for the next nine weeks.